

CO2 – AT2

Scenario Based Assessment

REPORT:

Finding the Closest Pair of Warehouses Using Brute Force Algorithm

Aim

To implement the brute-force algorithm to determine the closest pair of warehouse locations by computing the Euclidean distance between every possible pair.

Objective

- Understand the brute-force technique.
- Compute distances between all warehouse pairs.
- Identify the closest pair.
- Analyze time complexity.

Algorithm

1. Input the number of warehouses.
2. Read coordinates.
3. Initialize minimum distance.
4. Compare every pair.
5. Calculate Euclidean distance.
6. Update minimum distance when a smaller distance is found.
7. Display the closest pair.

Source Code and Output Screenshot:

```
main.c
1 #include <stdio.h>
2 #include <math.h>
3 int main()
4 {
5     int n, i, j;
6     float x[100], y[100];
7     float minDist, dist;
8     int p1 = 0, p2 = 1;
9     printf("Enter number of warehouses: ");
10    scanf("%d", &n);
11    printf("Enter coordinates (x y):\n");
12    for(i=0; i<n; i++)
13    {
14        scanf("%f %f", &x[i], &y[i]);
15    }
16    minDist = sqrt(pow(x[0]-x[1],2)+pow(y[0]-y[1],2));
17    for(i=0; i<n-1; i++)
18    {
19        for(j=i+1; j<n; j++)
20        {
21            dist = sqrt(pow(x[i]-x[j],2)+pow(y[i]-y[j],2));
22            if(dist < minDist)
23            {
24                minDist = dist;
25                p1 = i;
26                p2 = j;
27            }
28        }
29    }
```

Output

Enter number of warehouses: 5
Enter coordinates (x y):
2 3
12 30
40 50
5 1
12 10

Closest Warehouses:
Warehouse 1 and Warehouse 4
Minimum Distance = 3.61

=== Code Execution Successful ===

Brute Force Calculation (All Warehouse Pairs)

Given Warehouses

Warehouse Coordinates

W1	(2,3)
W2	(12,30)
W3	(40,50)
W4	(5,1)
W5	(12,10)

Distance Formula

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Pair 1: W1 – W2

$$\begin{aligned} &= \sqrt{(12 - 2)^2 + (30 - 3)^2} \\ &= \sqrt{10^2 + 27^2} \end{aligned}$$

$$\begin{aligned}
 &= \sqrt{100 + 729} \\
 &= \sqrt{829} \\
 &= 28.79
 \end{aligned}$$

Pair 2: W1 – W3

$$\begin{aligned}
 &= \sqrt{(40 - 2)^2 + (50 - 3)^2} \\
 &= \sqrt{38^2 + 47^2} \\
 &= \sqrt{1444 + 2209} \\
 &= \sqrt{3653} \\
 &= 60.44
 \end{aligned}$$

Pair 3: W1 – W4

$$\begin{aligned}
 &= \sqrt{(5 - 2)^2 + (1 - 3)^2} \\
 &= \sqrt{3^2 + (-2)^2} \\
 &= \sqrt{9 + 4} \\
 &= \sqrt{13} \\
 &= 3.61
 \end{aligned}$$

Current Minimum = 3.61

Pair 4: W1 – W5

$$\begin{aligned}
 &= \sqrt{(12 - 2)^2 + (10 - 3)^2} \\
 &= \sqrt{10^2 + 7^2} \\
 &= \sqrt{100 + 49} \\
 &= \sqrt{149} \\
 &= 12.21
 \end{aligned}$$

Minimum remains **3.61**

Pair 5: W2 – W3

$$\begin{aligned}
 &= \sqrt{(40 - 12)^2 + (50 - 30)^2} \\
 &= \sqrt{28^2 + 20^2} \\
 &= \sqrt{784 + 400} \\
 &= \sqrt{1184}
 \end{aligned}$$

$$= 34.41$$

Minimum remains **3.61**

Pair 6: W2 – W4

$$\begin{aligned} &= \sqrt{(5 - 12)^2 + (1 - 30)^2} \\ &= \sqrt{(-7)^2 + (-29)^2} \\ &= \sqrt{49 + 841} \\ &= \sqrt{890} \\ &= 29.83 \end{aligned}$$

Minimum remains **3.61**

Pair 7: W2 – W5

$$\begin{aligned} &= \sqrt{(12 - 12)^2 + (10 - 30)^2} \\ &= \sqrt{0 + 400} \\ &= \sqrt{400} \\ &= 20.00 \end{aligned}$$

Minimum remains **3.61**

Pair 8: W3 – W4

$$\begin{aligned} &= \sqrt{(5 - 40)^2 + (1 - 50)^2} \\ &= \sqrt{(-35)^2 + (-49)^2} \\ &= \sqrt{1225 + 2401} \\ &= \sqrt{3626} \\ &= 60.22 \end{aligned}$$

Minimum remains **3.61**

Pair 9: W3 – W5

$$\begin{aligned} &= \sqrt{(12 - 40)^2 + (10 - 50)^2} \\ &= \sqrt{(-28)^2 + (-40)^2} \\ &= \sqrt{784 + 1600} \\ &= \sqrt{2384} \end{aligned}$$

$$= 48.83$$

Minimum remains **3.61**

Pair 10: W4 – W5

$$\begin{aligned} &= \sqrt{(12 - 5)^2 + (10 - 1)^2} \\ &= \sqrt{7^2 + 9^2} \\ &= \sqrt{49 + 81} \\ &= \sqrt{130} \\ &= 11.40 \end{aligned}$$

Minimum remains **3.61**

Analysis

- Algorithm Used: Brute Force
- Total Comparisons:

$$\frac{n(n - 1)}{2}$$

For

$$n = 500$$

Comparisons

$$124750$$

Time Complexity

$$O(n^2)$$

Space Complexity

$$O(n)$$

Result

The brute-force algorithm successfully identifies the closest pair of warehouses by comparing every possible pair. Although it always produces the correct answer, its $O(n^2)$ time complexity causes performance to degrade as the number of warehouses increases. Therefore, it is suitable for small datasets but inefficient for large-scale logistics applications.